

VANILLA JAVA SCRIPT

Created: 2025-11-16 Sun 21:54

WHAT WE WILL COVER

WHAT WE WILL COVER

- Building an application using vanilla JavaScript and HTML

WHAT WE WILL COVER

- Building an application using vanilla JavaScript and HTML
- Creating DOM elements programmatically

WHAT WE WILL COVER

- Building an application using vanilla JavaScript and HTML
- Creating DOM elements programmatically
- Using the Document API to manipulate the DOM

WHY DO IT IN VANILLA JS?

WHY DO IT IN VANILLA JS?

- Understand the problem that we'll tackle (solve?)

WHY DO IT IN VANILLA JS?

- Understand the problem that we'll tackle (solve?)
- Do the frameworks job by ourselves.

WHY DO IT IN VANILLA JS?

- Understand the problem that we'll tackle (solve?)
- Do the frameworks job by ourselves.
- Suffer and appreciate the job that frameworks do.

OLD WAYS

- Applications used vanilla JavaScript and HTML along with CSS.
- jQuery used to interact with the DOM
 - browser differences
- Still a **viable** way to write applications

CLEANING UP!

build a simple application from scratch using only:

CLEANING UP!

build a simple application from scratch using only:

- vanilla JavaScript

CLEANING UP!

build a simple application from scratch using only:

- vanilla JavaScript
- HTML

CLEANING UP!

build a simple application from scratch using only:

- vanilla JavaScript
- HTML

Low abstraction level

CREATE A TODO APP

- **Main idea:** keep a list of the things I need to do (to-dos) in a day.
- A to-do can be marked as *done* so that it's removed from the list.
- A to-do can be *modified* when the user makes a typo or wants to change the description.

TODOS APPLICATIONS AND FEF

- classic example in frontend-framework world
- Hello world for FEF (see <https://todomvc.com>).

MOCKUP

My TODOs

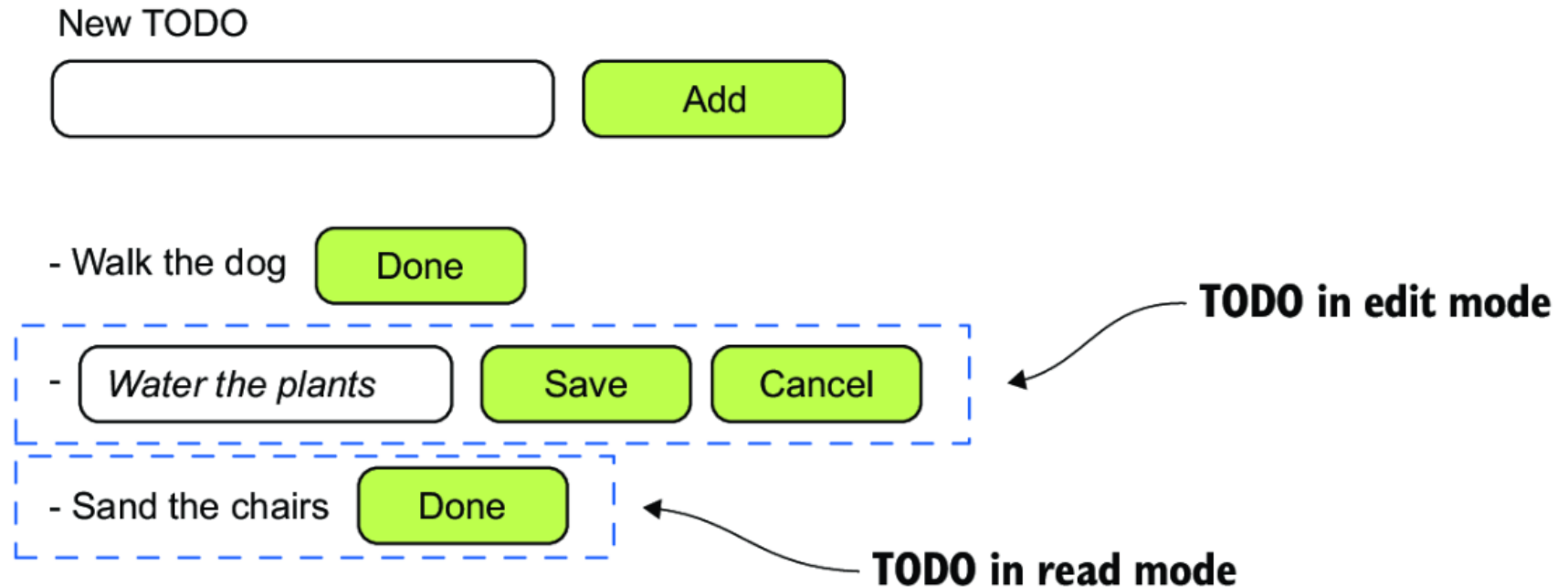


Figure 1: Wireframe design for the TODOs application

PLANING OUR APPLICATION

STATIC VS DYNAMIC CONTENT:

- static content
 - Title (My TODOs)
 - input field, label, Add button
- dynamically generated content based on the application state
 - TODO list items with the buttons

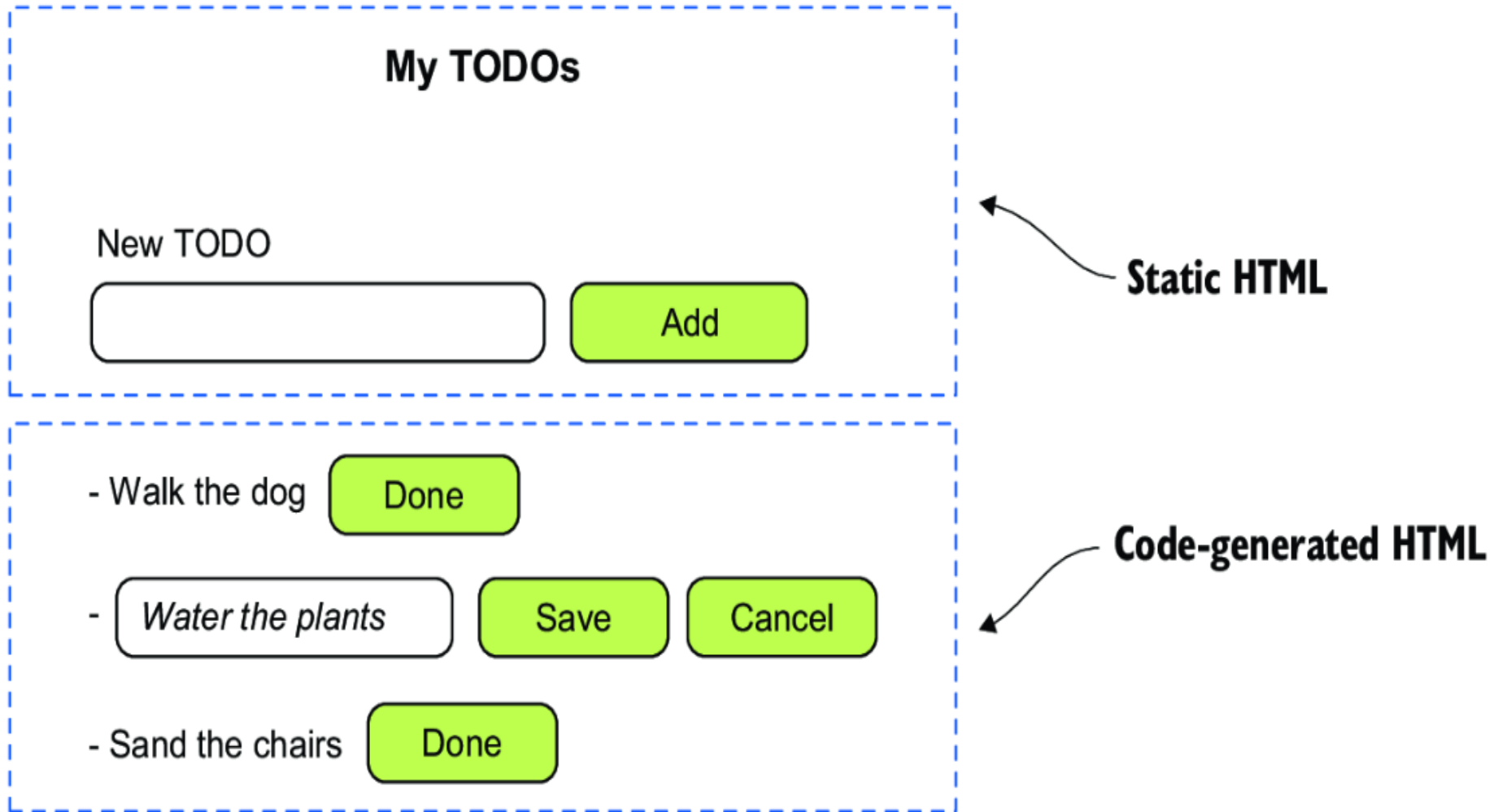


Figure 2: Static and dynamic HTML of the TODOs app

APPLICATION STATE

Dfn: The state is the information the application keeps track of that makes it look and behave the way it does at a particular moment.

Example: The application will look different when there are no to-dos than it does when there are to-dos.

APPLICATION BEHAVIOUR

- When the user writes a new to-do and clicks Add, the to-do is added to the list of to-dos.
- If the user presses the Enter key while the input field is focused, the to-do is appended to the list of to-dos as well.
- The application won't allow the user to add to-dos that are shorter than three characters.

To edit a to-do, the user has to double-click it.

If the user discards the changes by clicking the `Cancel` button, the to-do is restored to its previous state; the changes are lost.

When a to-do is marked as done, it's removed from the list of to-dos.

PROJECT SETUP

Two files:

- `todos.html` - HTML markup for the application
- `todos.js` - JavaScript code for the application

```
todo-vanilla/  
├── package.json  
├── todos.html  
└── todos.js
```


HTML MARKUP

1. load the `todos.js` as an ES module, `deferred` by default.
2. Create the `input` field, the field's `label`, and `Add button`
3. List where the to-dos will be rendered

The static HTML markup for the TODOs app

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>My TODOs</title>
    <script type="module" src="todos.js"></script>
  </head>
```

```
<body>
  <h1>My TODOs</h1>
  <div>
    <label for="todo-input">New TODO</label>
    <input type="text" id="todo-input" />
    <button id="add-todo-btn" disabled>Add</button>
  </div>
  <ul id="todos-list"></ul>
</body>
</html>
```



127.0.0.1:8080/examples/ch02/todos.html

My TODOs

New TODO

Add

BASIC STYLING

Add a `style.css` file

BASIC STYLING

Add a `style.css` file

- **Roboto** font

BASIC STYLING

Add a `style.css` file

- **Roboto** font
- Center the application horizontally.

BASIC STYLING

Add a `style.css` file

- **Roboto** font
- Center the application horizontally.
- Make the title bigger.

BASIC STYLING

Add a `style.css` file

- **Roboto** font
- Center the application horizontally.
- Make the title bigger.
- Place the input field's label above the input field and make it italic.

BASIC STYLING

Add a `style.css` file

- **Roboto** font
- Center the application horizontally.
- Make the title bigger.
- Place the input field's label above the input field and make it italic.
- Give the input and button some padding.

Adding style.css

```
<!DOCTYPE html>
<head>
  <meta charset="UTF-8" />
  <meta http-equiv="X-UA-Compatible" content="IE=edge" />
  <meta name="viewport" content="width=device-width,
                                initial-scale=1.0" />

  <link rel="stylesheet" href="styles.css" />
  <script type="module" src="todos.js"></script>

  <title>My TODOs</title>
</head>
```

Adding the Roboto font and centering the content inside `style.css`

```
@import url('https://fonts.googleapis.com/css2?family=Roboto:ital,w  
body {  
  font-family: 'Roboto', sans-serif;  
  width: 500px;  
  margin: 2em auto;  
}
```

Adding styles for the elements

```
label {  
  display: block;  
  font-style: italic;  
  margin-bottom: 0.25em;  
}  
h1 {  
  font-size: 3.5rem;  
}  
button,  
input {  
  font-family: inherit;  
  padding: 0.25em 0.5em;  
  font-size: 1rem;  
}
```

JAVASCRIPT CODE

JAVASCRIPT CODE

1. Define the application state
 - list of to-dos as an array of strings.

JAVASCRIPT CODE

1. Define the application state
 - list of to-dos as an array of strings.
2. Reference the DOM elements using the `document.getElementById()` function


```
// State of the app
const todos = ['Walk the dog', 'Water the plants',
               'Sand the chairs']
// HTML element references
const addTodoInput = document.getElementById('todo-input')
const addTodoButton = document.getElementById('add-todo-btn')
const todosList = document.getElementById('todos-list')
```

INITIALIZING THE VIEW

INITIALIZING THE VIEW

- dynamically generate the HTML markup depending on the application's state

INITIALIZING THE VIEW

- dynamically generate the HTML markup depending on the application's state
- attach event listeners to the DOM elements

Generate the HTML:

Generate the HTML:

- initialize the view

Generate the HTML:

- initialize the view
- iterate over the to-dos in the application's state

Generate the HTML:

- initialize the view
- iterate over the to-dos in the application's state
- **render**

Generate the HTML:

- initialize the view
- iterate over the to-dos in the application's state
- **render**

Dfn: To render means to transform some data into a visual representation — something we can see.

- `renderTodoInReadMode ( )` for viewing

- `renderTodoInReadMode ( )` for viewing
- append each element to the `<ul>` element.

EVENT LISTENERS

EVENT LISTENERS

- `<input>` field's input event.

EVENT LISTENERS

- `<input>` field's input event.
 - toggle the `disabled` attribute

EVENT LISTENERS

- `<input>` field's input event.
 - toggle the `disabled` attribute

New TODO

**With fewer than three characters,
the button remains disabled.**

New TODO

**With three characters or more,
the button is enabled.**

- listener to the `<input>` field's `keydown` event
- listener for the Add `<button>` element's `click` event.
 - call `addTodo ( )`,
 - clear the input field
 - and disable the Add button

The initialization of the application (todos.js)

```
// Initialize the view
for (const todo of todos) {
  todosList.append(renderTodoInReadMode(todo))
}

// input event on #todo-input field
addTodoInput.addEventListener('input', () => {
  addTodoButton.disabled = addTodoInput.value.length < 3
})

// event to filter the Enter key in the #todo-input
addTodoInput.addEventListener('keydown', ({ key }) => {
  if (key === 'Enter' && addTodoInput.value.length >= 3) {
    addTodo()
  }
})
```

The initialization of the application (todos.js)

```
// event for clicking the Add button
addTodoButton.addEventListener('click', () => {
  addTodo()
})

// Functions
function renderTodoInReadMode(todo) {
  // TODO: implement me!
}

function addTodo() {
  // TODO: implement me!
}
```

My TODOs

New TODO



'input' : () => {}



'keydown' : addTodo()

Listens to the changes in the field's value to enable/disable the button and to the Enter key being pressed to add a new to-do



'click' : addTodo()

Listens to the button being clicked to add a new to-do

Figure 3: The event listeners added to the HTML elements

Stylize the Add button

- Use **transition** CSS property.
- Use the **:disabled** & **:enabled** pseudoclasses.

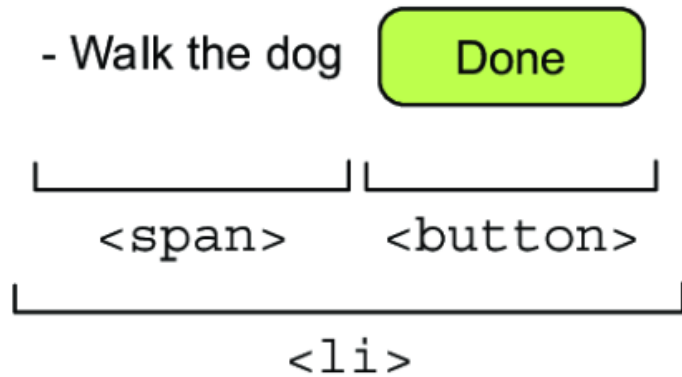
```
button:disabled {  
  background-color: darkgray;  
  color: lightgray;  
  transition: background-color 0.5s ease-in;  
  transition: color 0.5s ease-in;  
}
```

```
button:enabled {  
  background-color: #bdf4c;  
  color: black;  
  transition: background-color 0.5s ease-out;  
  transition: color 0.5s ease-out;  
}
```

RENDERING TO-DOS IN READ MODE

use the `document.createElement()` method of the Document API

- each to-do is inside a list item element (`<li>`).
- The to-do is a simple text in a `<span>` element.
- provide a button to mark a to-do as done.
- listener on the `<span>` for `dblclick` event
 - call `replaceChild()` on the `<li>`
 - render in edit mode (call `renderTodoInEditMode()`)



```
<li>  
  <span>Walk the dog</span>  
  <button>Done</button>  
</li>
```

Figure 4: A TODO in read mode

THE `replaceChild()` METHOD

- Replace the child node of a DOM node with another node. Parameters:
 - `newNode`—The node that replaces the old node
 - `oldNode`—The node to be replaced
- Event listener to the `<button>` element's `click` event
 - remove the to-do from the list (`removeTodo()` function)


```
function renderTodoInReadMode(todo) {  
  // A <li> element that contains the to-do  
  const li = document.createElement('li')  
  
  // A <span> with the to-do description  
  const span = document.createElement('span')  
  span.textContent = todo  
  
  // A dblclick event toggles the to-do to edit mode.  
  span.addEventListener('dblclick', () => {  
    const idx = todos.indexOf(todo)  
    // Replaces the to-do with its edit mode version  
    todosList.replaceChild(  
      renderTodoInEditMode(todo),  
      todosList.childNodes[idx]  
    )  
  })  
  li.append(span)
```

```
// A <button> to mark the to-do as done
const button = document.createElement('button')
button.textContent = 'Done'

// Removes the to-do from the list\
button.addEventListener('click', () => {
  const idx = todos.indexOf(todo)
  removeTodo(idx)
})
li.append(button)

return li
}
```

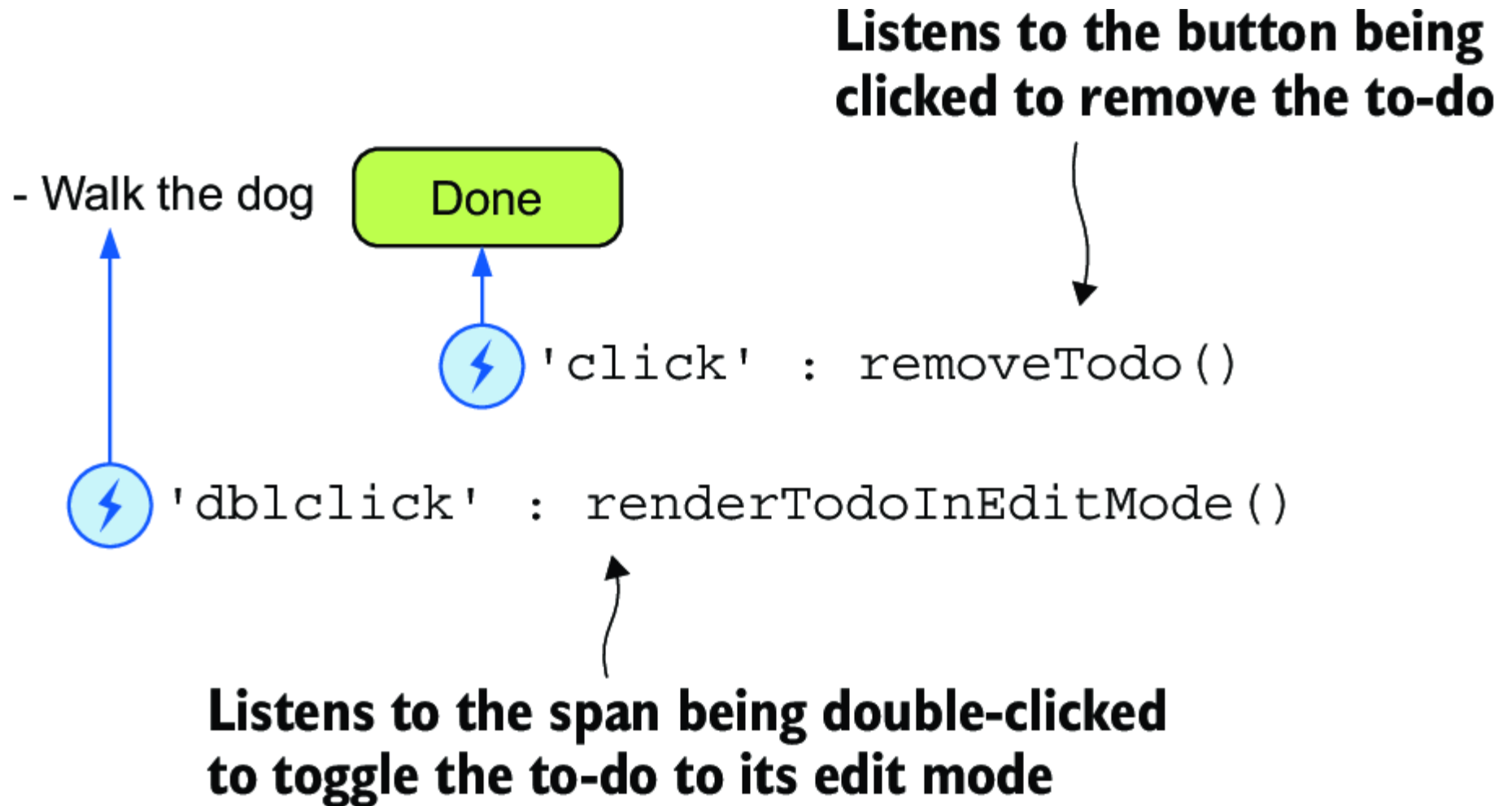


Figure 5: The event listeners added to the to-dos in read mode

RENDERING TO-DOS IN EDIT MODE

Figure 6: Todo in edit mode

RENDERING TO-DOS IN EDIT MODE

- `<li>` element with an `<input>` instead of a `<span>`

Figure 6: Todo in edit mode

RENDERING TO-DOS IN EDIT MODE

- `<li>` element with an `<input>` instead of a `<span>`
- two buttons:

Figure 6: Todo in edit mode

RENDERING TO-DOS IN EDIT MODE

- `<li>` element with an `<input>` instead of a `<span>`
- two buttons:
 - Save - calls `updateTodo ( )`

Figure 6: Todo in edit mode

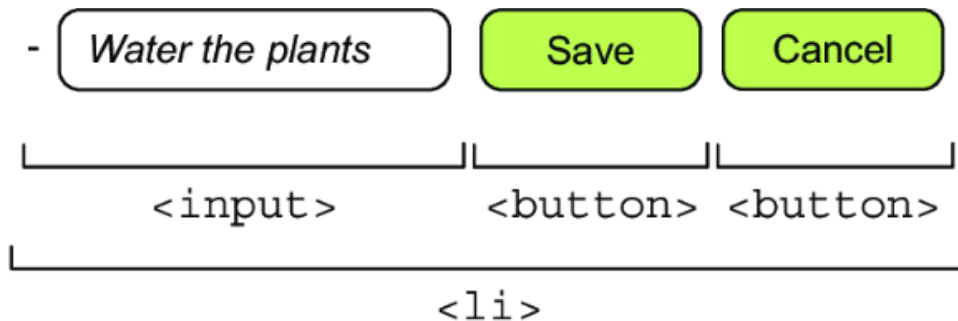
RENDERING TO-DOS IN EDIT MODE

- `<li>` element with an `<input>` instead of a `<span>`
- two buttons:
 - Save - calls `updateTodo()`
 - Cancel - calls `renderTodoInReadMode()`

Figure 6: Todo in edit mode

RENDERING TO-DOS IN EDIT MODE

- `<li>` element with an `<input>` instead of a `<span>`
- two buttons:
 - Save - calls `updateTodo()`
 - Cancel - calls `renderTodoInReadMode()`



```
<li>
  <input type="text" />
  <button>Save</button>
  <button>Cancel</button>
</li>
```

Figure 6: Todo in edit mode

```
function renderTodoInEditMode(todo) {  
  // <li> element that contains the to-do  
  const li = document.createElement('li')  
  
  // An <input> with the editable to-do description  
  const input = document.createElement('input')  
  input.type = 'text'  
  input.value = todo  
  li.append(input)  
  
  // A <button> to save the changes  
  const saveBtn = document.createElement('button')  
  saveBtn.textContent = 'Save'  
  
  // Updates the to-do description\\  
  saveBtn.addEventListener('click', () => {  
    const idx = todos.indexOf(todo)  
    updateTodo(idx, input.value)  
  })  
  li.append(saveBtn)
```

```

// A <button> to cancel the changes
const cancelBtn = document.createElement('button')
cancelBtn.textContent = 'Cancel'

// A click event cancels the changes.
cancelBtn.addEventListener('click', () => {
  const idx = todos.indexOf(todo)

  // Replaces the to-do with its read-mode version\
  todosList.replaceChild(
    renderTodoInReadMode(todo),
    todosList.childNodes[idx]
  )
})
li.append(cancelBtn)

return li
}

function updateTodo(index, description) {
  // TODO: implement me!
}

```

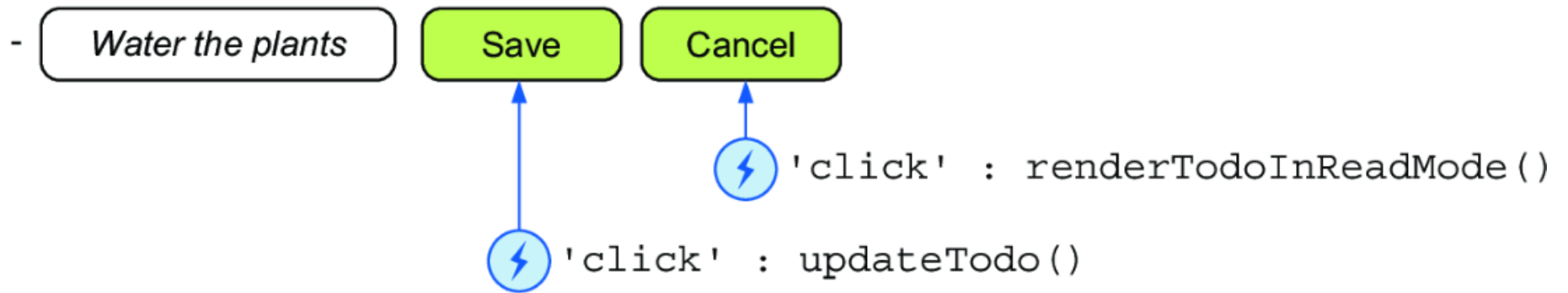


Figure 7: The events added to the to-dos in edit mode

ADDING TODOS

ADDING TODOS

- `addTodo ( )` read the description from the `<input>` element's `value` property

ADDING TODOS

- `addTodo ( )` read the description from the `<input>` element's `value` property
- Push into the array of to-dos.

ADDING TODOS

- `addTodo ( )` read the description from the `<input>` element's `value` property
- Push into the array of to-dos.
- Call the `renderTodoInReadMode ( )`

ADDING TODOS

- `addTodo ( )` read the description from the `<input>` element's `value` property
- Push into the array of to-dos.
- Call the `renderTodoInReadMode ( )`
- Clear the `<input>` element's `value` property

ADDING TODOS

- `addTodo ( )` read the description from the `<input>` element's `value` property
- Push into the array of to-dos.
- Call the `renderTodoInReadMode ( )`
- Clear the `<input>` element's `value` property
- Disable the Add button.

```
function addTodo() {  
  const description = addTodoInput.value  
  
  todos.push(description)  
  const todo = renderTodoInReadMode(description)  
  todosList.append(todo)  
  
  addTodoInput.value = ''  
  addTodoButton.disabled = true  
}
```

REMOVING TODOS

REMOVING TODOS

- Remove the `<li>` element from its parent `<ul>`

REMOVING TODOS

- Remove the `<li>` element from its parent `<ul>`
- call the `remove()` method on the target node

REMOVING TODOS

- Remove the `<li>` element from its parent `<ul>`
- call the `remove()` method on the target node

```
function removeTodo(index) {  
  todos.splice(index, 1)  
  todosList.childNodes[index].remove()  
}
```

UPDATE TODOS

UPDATE TODOS

- `updateTodo ( )` with two parameters:

UPDATE TODOS

- `updateTodo ( )` with two parameters:
 - the index of the to-do to update

UPDATE TODOS

- `updateTodo ( )` with two parameters:
 - the index of the to-do to update
 - the new description of the to-do.

UPDATE TODOS

- `updateTodo ( )` with two parameters:
 - the index of the to-do to update
 - the new description of the to-do.
- use `renderTodoInReadMode ( )` function to render the HTML

UPDATE TODOS

- `updateTodo ( )` with two parameters:
 - the index of the to-do to update
 - the new description of the to-do.
- use `renderTodoInReadMode ( )` function to render the HTML

```
function updateTodo(index, description) {  
  todos[index] = description  
  const todo = renderTodoInReadMode(description)  
  todosList.replaceChild(todo, todosList.childNodes[index])  
}
```

FURTHER ENHANCEMENTS

FURTHER ENHANCEMENTS

- Finished todos should be crossed out (not deleted)

FURTHER ENHANCEMENTS

- Finished todos should be crossed out (not deleted)
- Don't add duplicate todo (warn the user)

FURTHER ENHANCEMENTS

- Finished todos should be crossed out (not deleted)
- Don't add duplicate todo (warn the user)
- Read each todo item when added to the list.

SUMMARY

SUMMARY

- use of the Document API is the most burdensome to write.
 - abstract the manipulation of the DOM
 - focus on the application logic

SUMMARY

- use of the Document API is the most burdensome to write.
 - abstract the manipulation of the DOM
 - focus on the application logic
- mixing application logic and DOM manipulation

SUMMARY

- use of the Document API is the most burdensome to write.
 - abstract the manipulation of the DOM
 - focus on the application logic
- mixing application logic and DOM manipulation
- overly verbose
 - events that change the state must handle updates to the DOM.